Transactions in DBMS

Scenario-Based Practice Problems

CSE 2205 · Database Management System · Week 8

Instructions. Work through each problem on paper. Show every step: intermediate values, conflicting pairs, graphs and final answers. Part A contains the questions; Part B (starting on a new page) contains the worked solutions.
Notation: $R_i(X)$ = Ti reads X, $W_i(X)$ = Ti writes X, C_i = commit, A_i = abort.

PART A · Problems

Problem 1 — Interleaved Bank Schedule

Isolation violation · serial vs. concurrent results

Scenario. A = 1000 Tk and B = 2000 Tk. **T1** transfers 100 Tk from A to B. **T2** adds 10% interest to both accounts (A := A × 1.1, B := B × 1.1). The two transactions run in the following interleaving:

Step	T1	T2
1	R(A)	
2	A := A − 100; W(A)	
3		R(A)
4		A := A × 1.1; W(A)
5		R(B)
6		B := B × 1.1; W(B)
7	R(B)	
8	B := B + 100; W(B)	

(a) Find the final values of A and B for this schedule.

(b) Find the results of both serial orders (T1 then T2, and T2 then T1).

(c) Is the schedule correct? Draw the precedence graph and name the ACID property that is violated.

Problem 2 — Precedence Graph with Four Transactions

Conflict serializability · topological order

Scenario. Consider the following schedule S over items X, Y and Z:

Step	T1	T2	T3	T4
1	R(X)			
2		W(X)		
3			R(Y)	
4	W(Y)			
5				R(Z)
6			W(Z)	
7		W(Y)		

(a) List every edge of the precedence graph, grouped by data item.

(b) Is S conflict serializable? How many equivalent serial orders exist?

(c) What changes if the operation $W_4(X)$ is appended as step 8?

Problem 3 — Lock Table and Deadlock Detection

Exclusive locks · wait-for graph · victim selection

Scenario. Three transactions use exclusive locks only. T1 needs A then B; T2 needs B then C; T3 needs C then A. The *rollback cost* (log records written so far) is T1 = 3, T2 = 5, T3 = 4.

Step	Request	Result
1	T1: X(A)	?
2	T2: X(B)	?
3	T3: X(C)	?
4	T1: X(B)	?
5	T2: X(C)	?
6	T3: X(A)	?

(a) Fill in the Result column (granted / waits).

(b) Draw the wait-for graph. Is there a deadlock?

(c) Which victim should be chosen? Describe what happens next and give the equivalent serial order.

(d) Suggest a rule that prevents this deadlock.

Problem 4 — Recovery with a Checkpoint

Write-ahead log · redo / undo lists · final values

Scenario. Initial database values: A = 500, B = 300, C = 200. The log on disk at the time of a crash is:

```
<T1 start>
<T1, A, 500, 400>
<T2 start>
<T2, B, 300, 350>
<T1 commit>
<checkpoint {T2}>
<T3 start>
<T3, C, 200, 260>
<T2, A, 400, 450>
<T2 commit>
<T3, B, 350, 310>
--- system crash ---
```

(a) Build the redo list and the undo list. Explain why T1 is ignored.

(b) Find the final values of A, B and C after recovery.

(c) How much of the log must be examined, and why does the checkpoint help?

Problem 5 — PostgreSQL Isolation Levels with Numbers

READ COMMITTED vs. REPEATABLE READ · error 40001

Scenario. Table account holds A-101 = 1000 and B-202 = 2000. Session 1 is run twice: once at **READ COMMITTED** and once at **REPEATABLE READ**.

t	Session 1	Session 2
1	BEGIN [isolation level];	
2	SELECT SUM(balance), COUNT(*); → 3000, 2	
3		BEGIN; UPDATE account SET balance = balance − 300 WHERE acc_no = 'A-101'; INSERT ('C-303', 800); COMMIT;
4	SELECT SUM(balance), COUNT(*); → ?	
5	UPDATE account SET balance = balance + 100 WHERE acc_no = 'A-101'; → ?	
6	SELECT balance WHERE acc_no = 'A-101'; → ?	
7	COMMIT;	

(a) What is returned at steps 4, 5 and 6 under each isolation level?

(b) What are the final contents of the table under each level?

(c) What should the application do after an error at step 5?

PART B · Worked Solutions

Problem 1 — Interleaved Bank Schedule

Solution

(a) Trace the schedule.

Step	Operation	Value after
2	A := 1000 − 100	A = 900
4	A := 900 × 1.1	A = 990
6	B := 2000 × 1.1	B = 2200
8	B := 2200 + 100	B = 2300

Final: **A = 990, B = 2300, total = 3290**.

(b) Serial results. Correct total = 3000 + 10% interest = **3300** (the transfer does not change the total).

Order	A	B	Total
T1 then T2	(1000 − 100) × 1.1 = 990	(2000 + 100) × 1.1 = 2310	3300
T2 then T1	1000 × 1.1 − 100 = 1000	2000 × 1.1 + 100 = 2300	3300
Interleaved	**990**	**2300**	**3290**

(c) Verdict. The result (990, 2300) matches neither serial order, and 10 Tk has vanished. Conflicts: $W_1(A)$ before $R_2(A)$ gives **T1 → T2**; $W_2(B)$ before $R_1(B)$ gives **T2 → T1**. The precedence graph has a **cycle**, so the schedule is not conflict serializable. The **Isolation** property is violated, and the database ends in an inconsistent state.

Problem 2 — Precedence Graph with Four Transactions

Solution

(a) Edges, item by item.

Item	Conflicting pairs (first → second)	Edges
X	$R_1(X) \rightarrow W_2(X)$	T1 → T2
Y	$R_3(Y) \rightarrow W_1(Y)$; $R_3(Y) \rightarrow W_2(Y)$; $W_1(Y) \rightarrow W_2(Y)$	T3 → T1; T3 → T2; T1 → T2
Z	$R_4(Z) \rightarrow W_3(Z)$	T4 → T3

(b) The graph is T4 → T3 → T1 → T2, plus T3 → T2. There is **no cycle**, so S is conflict serializable. T4 has no incoming edge, so it goes first, then T3, then T1, then T2. The only topological order is **T4, T3, T1, T2**: exactly 1 equivalent serial schedule out of 4! = 24 possible serial orders.

(c) Appending $W_4(X)$ adds $R_1(X) \rightarrow W_4(X)$, giving **T1 → T4**, and $W_2(X) \rightarrow W_4(X)$, giving **T2 → T4**. Now T4 → T3 → T1 → T4 is a **cycle**, so the schedule is **not** conflict serializable.

Problem 3 — Lock Table and Deadlock Detection

Solution

(a) Lock table.

Step	Request	Result	Reason
1	T1: X(A)	Granted	A is free
2	T2: X(B)	Granted	B is free
3	T3: X(C)	Granted	C is free
4	T1: X(B)	**Waits**	B held by T2
5	T2: X(C)	**Waits**	C held by T3
6	T3: X(A)	**Waits**	A held by T1

(b) Wait-for graph. Edges: T1 → T2 (step 4), T2 → T3 (step 5), T3 → T1 (step 6). A cycle in the wait-for graph means **deadlock**.

(c) Victim selection. Choose the cheapest rollback: **T1** (cost 3 versus 5 and 4).

- T1 aborts and releases A.
- T3's pending request for A is granted. T3 finishes, commits and releases C and A.
- T2 now gets C, finishes, commits and releases B and C.
- T1 restarts and acquires A and B without conflict.

Equivalent serial order: **T3, T2, T1′** (T1′ = the restarted T1).

(d) Prevention. Impose a global lock order A < B < C. T3 would then request A before C, so it would wait while holding nothing, and no cycle can form. Other options: short transactions and `lock_timeout`.

Problem 4 — Recovery with a Checkpoint

Solution

(a) Classify each transaction.

Txn	Evidence in the log	Action
T1	Committed *before* the checkpoint, so its changes are already on disk	Ignored
T2	Active at the checkpoint and has <T2 commit>	**Redo**
T3	Started after the checkpoint, no commit record	**Undo**

(b) Apply recovery. Undo T3 in reverse order, then redo T2 forward.

Action	Item	Value set
Undo T3: <T3, B, 350, 310>	B	350
Undo T3: <T3, C, 200, 260>	C	200
Redo T2: <T2, B, 300, 350>	B	350
Redo T2: <T2, A, 400, 450>	A	450

Final: **A = 450, B = 350, C = 200**. Check: A = 500 − 100 + 50 = 450, and the total is 1000 both before and after.

(c) Only the records after the checkpoint (5 records) need examining, plus T2's earlier record back to <T2 start>. T1's records are skipped entirely. A checkpoint bounds recovery time because everything before it is already safely on disk.

Problem 5 — PostgreSQL Isolation Levels with Numbers

Solution

(a) Results.

Step	READ COMMITTED	REPEATABLE READ
4	Fresh snapshot: A = 700, B = 2000, C = 800 → **SUM = 3500, COUNT = 3**	Snapshot fixed at step 2 → **3000, 2**
5	Latest committed row is 700, so 700 + 100 = **800** (OK)	**ERROR 40001: could not serialize access due to concurrent update**
6	**800**	**ERROR: current transaction is aborted**
7	COMMIT succeeds	COMMIT is reported as **ROLLBACK**

(b) Final table.

Level	A-101	B-202	C-303	Total
READ COMMITTED	800	2000	800	**3600**
REPEATABLE READ (before retry)	700	2000	800	**3500**
REPEATABLE READ (after retry)	800	2000	800	**3600**

(c) Catch SQLSTATE 40001 and **retry the whole transaction** from BEGIN. The retry takes a new snapshot, sees A = 700, and updates it to 800.

Teaching point. The update is written as `balance = balance + 100`, which is safe at READ COMMITTED. If the application had instead read 1000 at step 2 and written back 1100, it would silently overwrite Session 2's −300: a **lost update**. Use a single-statement update or `SELECT … FOR UPDATE`.